

Unit – 2 : Section – 2 : Basics of C#

What is OOP?

Object-Oriented Programming (OOP) is a programming paradigm that organizes software design around classes and objects, rather than functions and logic. It focuses on binding data and the functions that operate on that data into a single unit. The goal of OOP is to represent real-world entities in your code, making programs modular, reusable, and easier to manage.

Advantages of OOP:

- **Modularity:** Code is organized into self-contained units (objects).
- **Reusability / Code Reusability:** Code can be reused through inheritance. Features like inheritance and polymorphism allow for code reuse, reducing the length and complexity of programs.
- **Maintainability:** Easier to debug and update code.
- **Security:** Data can be protected through encapsulation (**Encapsulation:** Hides internal data from unauthorized access, providing data security).
- **Clarity:** Provides a clear, organized structure for programs, making them easier to maintain and debug.
- **Scalability:** Allows developers to create large, complex, and scalable applications by breaking them down into smaller, manageable objects.

Classes and Objects:

- **Class:** A blueprint or template for creating objects. It defines the properties (data) and methods (functions) that objects of that class will have.
- **Object:** An object is an instance of a class.

Creating a Class and Object:

Example 1:

Creating a class:

```
// Define a class named 'Student'  
public class Student  
{
```

```

// Properties
public string Name { get; set; }
public int RollNumber { get; set; }

// Method
public void DisplayInfo()
{
    System.Console.WriteLine($"Name: {Name}, Roll Number: {RollNumber}");
}
}

```

Creating an object:

```

// In your main program, create an object (instance) of the Student class
Student student1 = new Student();

// Access and set properties
student1.Name = "Alice";
student1.RollNumber = 101;

// Call a method
student1.DisplayInfo(); // Output: Name: Alice, Roll Number: 101

```

Example 2:

Creating a class:

```

// Define a class
public class Car
{
    // Properties
    public string Model { get; set; }
    public int Year { get; set; }

    // Method
    public void Start()
    {
        Console.WriteLine($"{Model} starting...");
    }
}

```

```
}
```

Creating an object:

```
// Create an object  
Car myCar = new Car();  
myCar.Model = "Toyota";  
myCar.Year = 2023;  
myCar.Start();
```

Structure of a Class:

A class in C# can contain the following members:

- **Fields:** Variables that hold data.
- **Properties:** Provide controlled access to fields or a flexible way to access and modify private fields.
- **Methods:** Functions that define the behavior of the class or Functions that perform actions.
- **Constructors:** Special methods for initializing objects. It is called when an object is created, used for initialization.
- **Events:** Notifies other objects of significant changes.
- **Destructors (Finalizers):** Special methods that contain code to be executed when an object is being destroyed by the garbage collector. (Less common in C# due to garbage collection) Used for cleanup.

Data Types and Operators

Data Types:

C# supports various data types, including:

- **Value types:** Directly hold their data. Examples include `int`, `float`, `double`, `bool`, `char`, and `struct`.
- **Reference types:** Store a reference to the data, not the data itself. Examples include `class`, `string`, `array`, `interface`, `array`, and `delegate`.

Example:

```
// Value type
int age = 20;

// Reference type (string is a reference type)
string name = "Bob";

// Reference type (creating an object)
Student student2 = new Student();
```

Operators:

Operators are symbols that tell the compiler to perform specific mathematical or logical manipulations. C# provides arithmetic, relational, logical, assignment, and other operators.

Types of operators:

- **Arithmetic:** +, -, *, /, %
- **Relational:** ==, !=, >, <, >=, <=
- **Logical:** && (and), || (or), ! (not)
- **Assignment:** =, +=, -=, *=
- **Conditional (Ternary):** ? :

Example:

```
int x = 10, y = 5;
bool result = (x > y) && (x != 0); // result is true
int z = (x > y) ? x : y;           // z is 10
```

Example:

```
int a = 10, b = 5;
int sum = a + b; // Arithmetic
bool isEqual = (a == b); // Relational
bool condition = (a > 0 && b < 10); // Logical
```

Constructors and Destructors

Constructor:

A constructor is a special method used to initialize an object when it is created. It has the same name as its class and no return type.

Example:

```
public class Employee
{
    public string Name { get; set; }
    public int Id { get; set; }

    // Constructor
    public Employee(string name, int id)
    {
        this.Name = name;
        this.Id = id;
    }
}
```

Example:

```
public class Person
{
    public string Name { get; set; }
    public Person(string name) // Constructor
    {
        Name = name;
    }
}
Person p = new Person("Alice");
```

Destructor

A destructor (or finalizer) is a special method used to free resources or release unmanaged resources. In C#, the Garbage Collector (GC) handles memory management, so destructors are rarely needed. They are defined with a tilde (~) before the class name.

Example:

```

class MyClass
{
    // Destructor
    ~MyClass()
    {
        // Cleanup code
    }
}

```

Abstraction and Interface

Abstraction:

Abstraction is the process of hiding implementation details and showing only the functionality or essential features to the user. In C#, this is achieved using abstract classes and interfaces.

Abstract class example:

```

// Abstract class
public abstract class Shape
{
    // Abstract method (no body)
    public abstract double GetArea();

    // Concrete method (with body)
    public void Display()
    {
        System.Console.WriteLine($"This is a shape with area: {GetArea()}");
    }
}

// Derived class
public class Circle : Shape
{
    public double Radius { get; set; }
    public override double GetArea()
    {
        return 3.14 * Radius * Radius;
    }
}

```

```
}
```

Interface:

An interface is a blueprint for a class that contains only method signatures, properties, and events, but no implementation. A class that implements an interface must provide an implementation for all its members. In short, a contract that defines a set of methods and properties that a class must implement.

Example:

```
// Interface
public interface ILoggable
{
    void Log(string message);
}

// Class implementing the interface
public class FileLogger : ILoggable
{
    public void Log(string message)
    {
        // Code to write the message to a file
        System.Console.WriteLine($"Logging to file: {message}");
    }
}
```

Example:

```
public interface IShape
{
    double CalculateArea();
}

public class Circle : IShape
{
    public double Radius { get; set; }
    public double CalculateArea()
    {
        return Math.PI * Radius * Radius;
    }
}
```

```
}  
}
```

Polymorphism:

The ability of an object to take on many forms. Polymorphism means "many forms" and allows an object to take on many different forms. In C#, it can be achieved through method overloading and method overriding.

- **Overloading / Method Overloading:** Method overloading allows a class to have multiple methods with the same name but different parameters (either in number, type, or order). (Defining multiple methods with the same name but different parameters within the same class.)

Example:

```
public class Calculator  
{  
    public int Add(int a, int b) { return a + b; }  
    public double Add(double a, double b) { return a + b; }  
    public int Add(int a, int b, int c) { return a + b + c; }  
}
```

Example:

```
public class Calculator  
{  
    public int Add(int a, int b) { return a + b; }  
    public double Add(double a, double b) { return a + b; }  
}
```

- **Overriding / Method Overriding:** Method overriding allows a derived class to provide a specific implementation for a method that is already defined in its base class. This is done using the `virtual` and `override` keywords. (Providing a specific implementation of a method in a derived class that is already defined in its base class. Requires `virtual` in base and `override` in derived.)

Example:

```
public class Animal
```



```

{
    public virtual void MakeSound()
    {
        System.Console.WriteLine("Some generic sound");
    }
}

public class Dog : Animal
{
    public override void MakeSound()
    {
        System.Console.WriteLine("Woof!");
    }
}

```

Example:

```

public class Animal
{
    public virtual void MakeSound() { Console.WriteLine("Generic animal sound"); }
}

public class Dog : Animal
{
    public override void MakeSound() { Console.WriteLine("Woof!"); }
}

```

Inheritance

Inheritance allows a new class (derived class) to inherit members from an existing class (base class), promoting code reuse and establishing a hierarchical relationship. (A mechanism where a new class (derived class) inherits properties and methods from an existing class (base class).)

Example:

```

// Base class
public class Person
{
    public string FirstName { get; set; }
    public string LastName { get; set; }
}

```

```
// Derived class
public class Manager : Person
{
    public int TeamSize { get; set; }
}
```

Example:

```
public class Vehicle // Base class
{
    public string Brand { get; set; }
}

public class Car : Vehicle // Derived class
{
    public string Model { get; set; }
}
```

Garbage Collection:

Garbage collection (GC) is a process in C# that automatically reclaims memory occupied by objects that are no longer in use. It prevents memory leaks by handling memory management automatically.

How it works:

1. **Marking:** The GC identifies all currently "live" objects that are still being used.
2. **Relocating:** The GC updates the references of the live objects.
3. **Compacting:** The GC reclaims the memory used by dead objects and compacts the live objects together, eliminating fragmentation.

Automatic memory management in C#. The .NET runtime automatically reclaims memory occupied by objects that are no longer referenced.

Arrays, collections, and indexers:

Jagged Array:

A jagged array is an "array of arrays" where each inner array can have a different dimension and size.

Example:

```
// Declare a jagged array of 3 rows
int[][] jaggedArray = new int[3][];

// Initialize each inner array with a different size
jaggedArray[0] = new int[] { 1, 2, 3 };
jaggedArray[1] = new int[] { 4, 5, 6, 7 };
jaggedArray[2] = new int[] { 8, 9 };

// Accessing an element
System.Console.WriteLine(jaggedArray[1][2]); // Output: 6
```

An array of arrays where each inner array can have a different size.

Code

```
int[][] jaggedArray = new int[3][];
jaggedArray[0] = new int[] { 1, 2, 3 };
jaggedArray[1] = new int[] { 4, 5 };
jaggedArray[2] = new int[] { 6, 7, 8, 9 };
```

Collections (Hashtable and ArrayList)

The `System.Collections` namespace provides several collection classes for handling data dynamically. Classes designed to store and manage groups of objects.

ArrayList

`ArrayList` is a dynamic array that can store elements of different type and grows automatically as elements are added. (Generally replaced by `List<T>` in modern C#).

Example:

```
using System.Collections;
ArrayList list = new ArrayList();
list.Add(10);
list.Add("Hello");
list.Add(true);
```

Hashtable

Hashtable stores key-value pairs. It uses a hash code to store and retrieve values, providing fast lookup times. (Generally replaced by Dictionary<TKey, TValue>).

Example:

```
using System.Collections;
Hashtable ht = new Hashtable();
ht.Add("ID", 101);
ht.Add("Name", "John Doe");
```

Indexer (One Dimension) and Property

Indexer:

An indexer allows an object to be indexed like an array. This lets you access data using the [] operator. Allows objects to be indexed like arrays.

Example:

```
public class MyData
{
    private string[] _data = new string[5];

    public string this[int index]
    {
        get { return _data[index]; }
        set { _data[index] = value; }
    }
}
```

Example:

```
public class MyCollection
{
    private string[] data = new string[10];
    public string this[int index] // Indexer
    {
        get { return data[index]; }
        set { data[index] = value; }
```

```

    }
}
MyCollection mc = new MyCollection();
mc[0] = "First Item";

```

Property:

A property provides controlled access to a class's internal fields, using get and set accessors. Provides a flexible mechanism to read, write, or compute the value of a private field.

Example:

```

public class Product
{
    private double _price;
    public double Price
    {
        get { return _price; }
        set
        {
            if (value > 0)
            {
                _price = value;
            }
        }
    }
}

```

Example:

```

public class Product
{
    private decimal _price;
    public decimal Price // Property
    {
        get { return _price; }
        set { if (value >= 0) _price = value; }
    }
}

```

Delegates:

A delegate is a type-safe function pointer that holds a reference to a method and can be used to invoke them.

Example:

```
// Define a delegate
public delegate void MyDelegate(string message);

// Method to be referenced by the delegate
public void ShowMessage(string msg)
{
    System.Console.WriteLine(msg);
}

// Instantiate the delegate
MyDelegate del = new MyDelegate(ShowMessage);
del("Hello from delegate");
```

Multicast delegates and events:

A multicast delegate can reference multiple methods. Events, built on delegates, provide a controlled way for a class to notify other objects of an action. (A delegate can hold references to multiple methods, and when invoked, all referenced methods are called.)

Example:

```
// Define a class with an event
public class Notifier
{
    public event MyDelegate OnNotify;

    public void RaiseEvent()
    {
        // Check if event has subscribers before invoking
        OnNotify?.Invoke("Event triggered!");
    }
}
```

```
// In your main method  
Notifier notifier = new Notifier();  
notifier.OnNotify += ShowMessage; // Subscribe a method to the event  
notifier.RaiseEvent();
```

Example:

```
public delegate void MyDelegate(string message);  
  
public class Publisher  
{  
    public event MyDelegate MyEvent;  
    public void RaiseEvent(string msg)  
    {  
        MyEvent?.Invoke(msg); // Raise the event  
    }  
}  
  
public class Subscriber  
{  
    public void HandleEvent(string message)  
    {  
        Console.WriteLine($"Event handled: {message}");  
    }  
}
```

```
// In your application:  
Publisher pub = new Publisher();  
Subscriber sub = new Subscriber();  
pub.MyEvent += sub.HandleEvent; // Subscribe  
pub.RaiseEvent("Hello Event!");
```

Event:

A mechanism for a class to notify other classes or objects when something interesting happens. Events are built upon delegates.

Exception handling

Exception handling is used to manage errors that occur during program execution. It is implemented using `try`, `catch`, and `finally` blocks. (A mechanism to handle runtime errors gracefully using `try`, `catch`, `finally` blocks.)

Example:

```
try
{
    int[] numbers = { 1, 2, 3 };
    System.Console.WriteLine(numbers[5]); // This will cause an IndexOutOfRangeException
}
catch (System.IndexOutOfRangeException ex)
{
    System.Console.WriteLine("An array index error occurred: " + ex.Message);
}
finally
{
    System.Console.WriteLine("This block always executes.");
}
```

Example:

```
try
{
    int result = 10 / 0; // Throws DivideByZeroException
}
catch (DivideByZeroException ex)
{
    Console.WriteLine($"Error: {ex.Message}");
}
finally
{
    Console.WriteLine("Execution finished.");
}
```


String handling

C# provides rich functionality for manipulating strings. The `string` type is immutable, but the `StringBuilder` class can be used for dynamic string modifications. Working with strings, including concatenation, manipulation, searching, and formatting.

Example:

```
string name = "Alice";
string newName = name.ToUpper(); // Returns "ALICE"

// Using StringBuilder for performance in loops
using System.Text;
StringBuilder sb = new StringBuilder();
for (int i = 0; i < 5; i++)
{
    sb.Append(i);
}
string result = sb.ToString(); // "01234"
```

Example:

```
string name = "Alice";
string greeting = "Hello, " + name + "!";
string formatted = $"Name: {name}, Length: {name.Length}";
```

Functions (methods)

A function (or method in a class) is a block of code that performs a specific task.

Creating functions with different types:

- **Value parameters:** A copy of the argument is passed.
- **Reference parameters (`ref`):** A reference to the argument is passed. Changes affect the original variable.
- **Output parameters (`out`):** Similar to `ref`, but the variable does not need to be initialized before passing.

- **Parameter array (`params`)**: Used to pass a variable number of arguments.

Example:

```
public void PassByValue(int x) { x = x + 1; }  
public void PassByRef(ref int x) { x = x + 1; }  
public void PassByOut(out int x) { x = 10; }  
public int Sum(params int[] numbers)  
{  
    int sum = 0;  
    foreach (int num in numbers) { sum += num; }  
    return sum;  
}
```